A Streaming Platform for Real-Time Log Anomaly Detection with Explainable Alerts

Pranshu Thakkar, Ritwik Gupta, Zaid Khan, Saatvik Das, Vedula Srisaket Rama
DevOps / Platform Engineering
[Institution / Organization]

***Abstract*—Production services emit millions of log lines a day, yet most alerting still relies on static, hand-tuned thresholds that are noisy at peak load, blind at quiet hours and unable to recognise failure modes nobody anticipated. This paper presents the design, implementation and evaluation of a streaming log anomaly detection platform that ingests structured logs through Apache Kafka, redacts personal data, mines log templates with Drain, aggregates one-minute behavioural features per service and scores every window with a three-tier ensemble: robust statistical baselines with a drift detector, a per-group isolation forest, and a DeepLog-style LSTM over request traces, complemented by hard rules. An alert router de-duplicates, groups related services into incidents and attaches an explanation to every alert. On a simulated five-service e-commerce workload with labelled fault injection covering six failure types, the platform detected all 12 injected incidents with an incident-level precision of 0.91 (F1 0.95), raised 2.3 alerts per service per day and kept the 95th-percentile log-to-score latency at 37.6 s, whereas the static rule in use today detected none of the incidents. A second, independent run with 18 incidents reproduced these results (precision 0.89, recall 1.00). We also report fifteen defects that surfaced only when the full containerised stack was run, and the lessons they hold for deploying such pipelines.**

***Index Terms*—log analysis, anomaly detection, stream processing, Apache Kafka, isolation forest, LSTM, AIOps, site reliability engineering.**

I. INTRODUCTION

Logs are the most detailed record of what a software service is doing, but in most organisations they are consulted only after users report a failure. Alerting on logs typically means static rules such as “more than 50 errors in 5 minutes”. Such rules suffer from three well-known problems [10], [11]: they do not adapt to diurnal and weekly traffic patterns, so they fire constantly at peak and stay silent at night; they only recognise the failures someone wrote a rule for; and they produce duplicate, context-free alerts that slow down responders and erode trust in alerting.

This work builds and evaluates a platform that replaces such thresholds with learned behaviour. Its goals, set out in the project proposal, are to centralise logs in a durable, replayable stream, to detect anomalous behaviour within 60 s of the log event (95th percentile), to catch failure modes nobody wrote a rule for with at least 0.80 precision and 0.70 recall, to keep alerts below five per service per day with an explanation attached to each, and to learn from responder feedback.

The contributions of this paper are:

- an end-to-end streaming architecture (Kafka, stream jobs with event-time windows and per-source watermarks, ClickHouse, object storage) that can be replayed and runs both as a single process and as a 20-service container stack;
- a label-free, three-tier detection ensemble with explanations, a feedback loop and a gated champion/challenger model lifecycle;
- an incident-level evaluation with labelled fault injection that compares the platform with the static rules in use today;
- an account of the defects found when the full stack was deployed, which are instructive for anyone operating similar pipelines.

II. RELATED WORK

Log parsing. Automated log analysis usually begins by separating the constant part of a message (the template) from its variables. Drain [1] does this online with a fixed-depth parse tree and is among the most accurate and efficient parsers in the LogPAI benchmark of Zhu *et al.* [6]; we use it for that reason.

Log-based anomaly detection. Early work mined console logs with PCA over message-count vectors [5]. He *et al.* [4] compared supervised and unsupervised detectors and found that unsupervised methods trade some accuracy for not needing labels, which most organisations lack. DeepLog [2] models the sequence of log keys with an LSTM and flags keys outside the top-k predictions; our third tier follows this design. A broad survey of the field is given in [10].

Outlier detection and robust statistics. The isolation forest [3] isolates anomalies with random axis-parallel splits and needs no labels; robust location and scale estimators such as the median absolute deviation [9] keep baselines from being distorted by the very outliers they are meant to find.

Streaming infrastructure. Kafka [7] provides the durable, partitioned log that decouples our stages, and the event-time windowing with watermarks we use follows the Dataflow model [12] as implemented in Apache Flink [8]. Model versions are tracked with MLflow [13].

Compared with prior work, which mostly evaluates detectors offline on fixed datasets, this project integrates parsing, multi-tier detection, explanation, alert routing and the model lifecycle in one operational system and measures it end to end, at the incident level, against the alerting it would

replace.

III. SYSTEM ARCHITECTURE

The platform is a five-stage streaming pipeline (Fig. 1). Every hop between stages is a Kafka topic, which decouples the components, absorbs bursts and allows history to be replayed whenever a parser or model changes. Each stage is a consumer group that commits offsets only after its output has been flushed, giving at-least-once delivery; the router de-duplicates downstream.

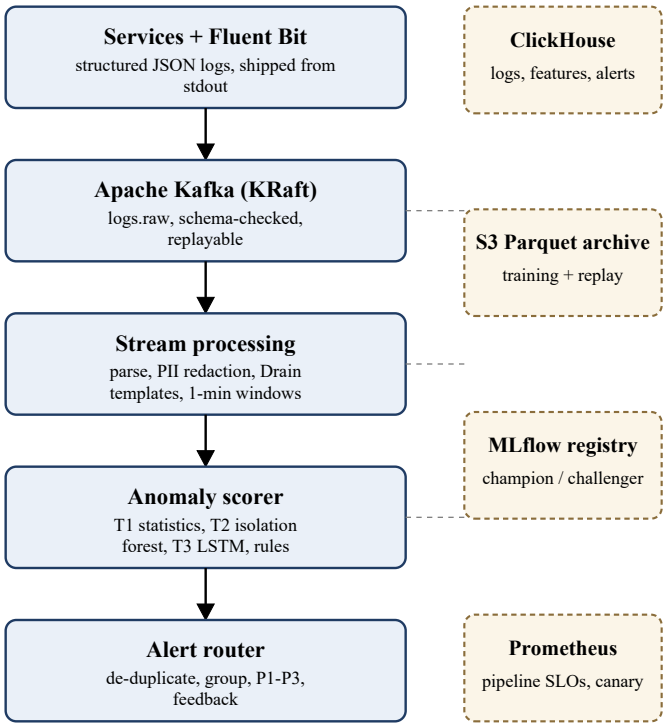


Fig. 1. System architecture. Solid arrows show the real-time path through Kafka topics; dashed boxes are the storage, training and observability services.

A. Ingestion

Services log one JSON object per line to standard output through a shared library (Python and Go) that enforces a common envelope (*ts, service, env, level, msg*, plus optional *trace_id* and attributes such as latency and HTTP status). Fluent Bit ships the lines to the *logs.raw* topic keyed by service, so a service's records stay ordered within a partition. Kafka runs in KRaft mode with producers using *acks=all* and idempotence; topics carry 72 h of history.

B. Stream Processing

The *parse-enrich* job validates each record against the envelope (invalid or oversized records go to a dead-letter topic with a reason), parses legacy plain-text formats with regex profiles, normalises timestamps, redacts personal data (e-mail addresses, card-like numbers, IP addresses and tokens) before anything is stored, and assigns a Drain template identifier. Template identifiers are derived from the service and the template text, so they are stable across restarts.

The *window-features* job aggregates one-minute tumbling event-time windows per service. The watermark is the minimum, over active sources, of each source's newest event time minus a 5 s bound on disorder; a source more than 10 s behind the newest is treated as idle so that a silent service cannot stall the watermark. Records up to 30 s late update their window as a revision. Crucially, a service that sends nothing still receives a feature vector with a growing *seconds-since-last* value, which is how silence becomes observable. A session window keyed by *trace_id* (30 s gap) emits each request's ordered template sequence for the sequence model.

C. Storage and Lifecycle

ClickHouse holds redacted logs (14-day TTL), templates, feature vectors (90 days), anomaly scores and alerts; sinks buffer rows and insert in large batches. Every topic is also archived hourly as Parquet in S3-compatible object storage for training and replay. A nightly job trains new models and registers them in MLflow.

D. Alert Routing

The router turns scores into a small number of notifications. Anomalies on the same service within 15 min join one open alert; alerts on services that call each other within 2 min share an incident identifier. Severity follows a routing policy kept in version control: P1 (score ≥ 0.9, or a hard rule on a tier-1 service) pages on-call, P2 goes to the team channel and P3 is dashboard-only. Delivery runs through Prometheus Alertmanager, and each alert carries *Real issue / False alarm* buttons whose verdicts tune per-service thresholds weekly and label the next training run.

IV. ANOMALY DETECTION METHOD

Because labelled incidents are rare, every tier is unsupervised. Each tier yields a score in [0, 1] for each service-minute window.

TABLE I
FEATURE VECTOR PER SERVICE AND MINUTE

Group	Features
Volume	total count; change vs. same minute last week; distinct hosts
Severity	error and warning counts; error ratio; fatal flag
Content	unique templates; templates new in 7 days; template-distribution entropy; 64-bucket hashed template histogram
Performance	p50 / p95 / p99 latency; ratio of 5xx responses
Liveness	seconds since the service's last record

A. Tier 1: Robust Statistical Baseline

For each service s, hour of week h and feature f, a rolling sample of recent *normal* values yields the median m and median absolute deviation (MAD). A window is scored with the robust z-score

$$z_f = (x_f - m_{s,h,f}) / (1.4826 \cdot \max(\mathrm{MAD}_{s,h,f}, \delta_f)) \quad (1)$$

where the per-feature floor δ keeps near-constant features from producing huge scores. Slots with fewer than 20 samples fall back to hour-of-day and then service-wide statistics, and only the direction that indicates trouble counts (e.g., rising latency, falling host count). Slow drifts such as memory leaks are caught by an exponentially weighted moving average of the signed z-score,

$$e_t = (1 - \alpha)\, e_{t-1} + \alpha\, z_t, \quad z_{\text{trend}} = (e_t - \mu_e) / \sigma_e \tag{2}$$

with $\alpha = 0.1$, where the mean and standard deviation of the EWMA are calibrated out of sample on the last two days of history. The EWMA is not updated on windows that are already acutely anomalous ($|z| \geq 6$), so a short outage does not leave a long tail. The tier score maps the strongest deviations through a logistic function centred at $z = 4.5$.

B. Tier 2: Isolation Forest

One isolation forest [3] (100 trees) per service group scores the full vector of scalar features, each robust-scaled per service so a single model can serve services of very different volume. The 64 hashed template buckets are summarised by one feature, the Jensen–Shannon divergence [14] between the window's template distribution p and the service's normal distribution q:

$$\mathrm{JS}(p \,\|\, q) = \tfrac{1}{2}\,\mathrm{KL}(p \,\|\, m) + \tfrac{1}{2}\,\mathrm{KL}(q \,\|\, m), \quad m = \tfrac{1}{2}(p + q) \tag{3}$$

Raw forest scores are calibrated against the training distribution and exported to ONNX for serving, with a parity check against the original model. Since a forest cannot rank a point beyond its training range above the range's edge, single-feature extremes are left to Tier 1 and Tier 2 contributes unusual *combinations* of individually plausible values.

C. Tier 3: Sequence Model

Following DeepLog [2], an LSTM (embedding 32, two layers of 64 units) predicts the next *service:template* token of a trace from the previous four; a token outside the top-3 predictions is a violation. Only the first violation of a trace counts, and it is attributed to the service whose step was *expected*: after one deviation every later step looks wrong, and a missing step (for example, a skipped payment) is the absent service's problem. The tier score is a binomial test of the share of violating traces against the service's normal violation rate.

D. Hard Rules, Ensemble and Debounce

Two high-precision rules complement the models: a template first seen within ten minutes at ERROR or FATAL level occurring at least three times, and a service that stops logging for more than 90 s. Tier scores are combined as a weighted mean over the tiers present,

$$S = \max\left(\Sigma_i\, w_i s_i \,/\, \Sigma_i\, w_i,\ 0.85 \cdot \max_i s_i \cdot \mathbb{1}[\max_i s_i \geq 0.9] \right) \tag{4}$$

so that one very confident tier is not averaged away. An alert fires when S exceeds the threshold in two of the last three windows; hard rules bypass this debounce. Every anomaly records its top three deviating features, the templates whose counts moved most and five redacted sample lines.

E. Model Lifecycle

A nightly job trains on the last 14 days (7 in our experiments), holds out the most recent 20 % and adds synthetic anomalies (volume drops and spikes, error bursts, latency inflation, new templates, template-mix shifts, host loss). It tunes the ensemble weights and threshold for F1 under an alert budget of five per service per day, and promotes a model only if F1 does not regress against the champion on the same evaluation set and the projected alert rate stays within budget. New models shadow the champion before promotion, scorers hot-reload the champion, and a one-command rollback restores the previous version. Drift is monitored with the population stability index.

V. IMPLEMENTATION

The platform is written in Python (about 7,000 lines plus tests). The stream jobs are implemented once and run in two modes: a single-process *lite* stack over an in-memory partitioned log and SQLite, used for evaluation and demonstrations, and a container stack in which each stage is a separate consumer group over Kafka with ClickHouse storage (Table II). PyFlink and Flink SQL versions of the stream jobs, a Helm chart (Strimzi, KEDA autoscaling on consumer lag, NetworkPolicies, External Secrets), Argo CD manifests, Terraform modules and CI workflows (lint, type and unit tests, schema compatibility, image scanning and signing, integration tests) are included for production deployment.

TABLE II
TECHNOLOGY STACK OF THE CONTAINER DEPLOYMENT

Concern	Choice
Messaging	Apache Kafka 4.3 (KRaft), Schema Registry
Shipping	Fluent Bit 3.1
Analytics store	ClickHouse 24.8 (MergeTree, TTLs)
Object storage	S3 API (SeaweedFS locally)
Models	scikit-learn, ONNX Runtime, PyTorch, MLflow 2.17
Observability	Prometheus, Alertmanager, Grafana
Deployment	Docker Compose; Helm, Argo CD, Terraform

The test suite contains 68 unit tests, two end-to-end tests that simulate traffic, train a model, inject faults and assert explained alerts, and integration tests against live Kafka and ClickHouse.

VI. EVALUATION

A. Workload and Fault Injection

Real incident labels were not available, so the evaluation uses a simulated e-commerce system of five services (gateway, orders, payments, inventory, notifications) with 2–3 hosts each. Requests form traces across services, traffic follows daily and weekly seasonality, and the logs include background noise, benign deployments and realistic personal data. A fault-injection API introduces the six labelled failure types of Table III, each of which doubles as ground truth.

TABLE III
INJECTED FAULT TYPES

Fault	Effect in the logs
Downstream timeout	payment calls to the bank gateway time out; retries, 504s, latency
Crash loop	hosts restart repeatedly; liveness gaps, fatal exits
Bad deploy	a new exception after a release
Silent service	a service stops logging entirely
Memory leak	latency and GC warnings grow over two hours
Skipped step	orders confirmed without the payment step

Each experiment replays 7 days of normal history (about 1.2 million log lines), trains a model with the lifecycle of Section IV-E, and then replays an evaluation period with injected incidents spread over the day, including the low-traffic night hours.

B. Metrics and Baselines

Metrics are computed per incident. An incident affects its service and, for faults that propagate, the services that call it. An alert is a true positive if it opens on an affected service between one minute before the fault and five minutes after it ends; otherwise it is a false positive. Recall is the share of incidents detected, precision is the share of alerts that are true positives, and detection delay is the time from fault start to the first matching alert. Latency is measured from the newest log event in a window to the emission of its score. The baselines are the static rule in use today (more than 50 errors in 5 min per service) and a five times more sensitive variant (more than 10), both de-duplicated like the platform's alerts.

C. Results

Table IV reports the main experiment (2 evaluation days, 12 incidents, two of each type) and an independent repetition with different traffic and incident schedules (3 days, 18 incidents).

TABLE IV
INCIDENT-LEVEL RESULTS

Metric	Platform	2nd run	>50/5m	>10/5m
Precision	0.91	0.89	–	1.00
Recall	1.00	1.00	0.00	0.33
F1	0.95	0.94	0.00	0.50
Alerts (false)	23 (2)	36 (4)	0 (0)	9 (0)
Alerts/service/day	2.3	2.4	0	0.9
Median delay (s)	183	155	–	284
Latency p95 (s)	37.6	40.9	–	–

The platform detected every injected incident in both runs, at a precision above the 0.80 target and well under the budget of five alerts per service per day; the 95th-percentile latency stayed below the 60 s target. The static rule in use today never fired: at the simulated traffic levels most failures do not produce 50 errors in five minutes, which illustrates the "blind at quiet times" problem. The sensitive rule was precise but caught only a third of the incidents, later than the platform. Fig. 2 breaks recall down by fault type.

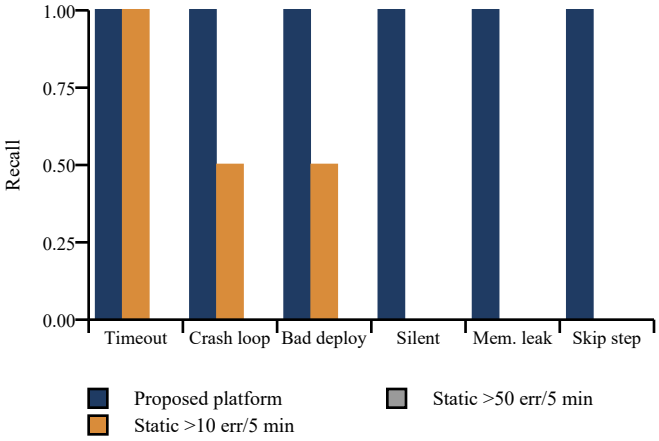


Fig. 2. Recall by fault type in the main experiment (two incidents per type). Static rules see only error counts and miss silent services, slow leaks and skipped workflow steps.

The median detection delay was 2.3–4.4 min for acute faults, which reflects one-minute windows plus the two-of-three debounce. The memory leak was detected after 19.7 min on average, while latency was still climbing, by the drift detector; the skipped step was detected after 10.7 min by the sequence model, the only tier that can see it. Window-level evaluation during training, against the holdout with synthetic anomalies, gave a precision of 0.85 and recall of 0.85.

D. Live Demonstration

The platform also runs in real time with a dashboard and a fault-injection control. In a recorded run, a bank-gateway timeout injected into the payments service raised a P1 alert for payments and P2 alerts for the dependent gateway and orders services, grouped as one incident, 138 s after injection, with a log-to-score latency of 27 s. Fig. 3 shows the explanation attached to the alert: without any rule naming the failure, the platform identified the retry and timeout templates of the bank-gateway call and attached redacted sample lines.

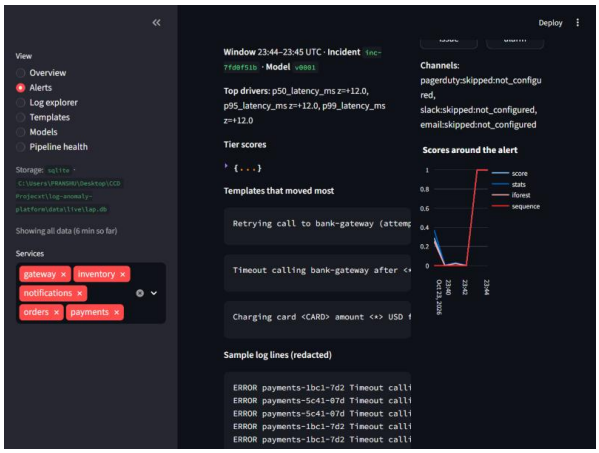


Fig. 3. Alert detail from the live demonstration: top drivers, the templates that moved most (bank-gateway retries and timeouts), redacted sample lines and the score history around the alert.

E. Development History

The evaluation harness was run after every change, which exposed several design errors (Table V). The first version's drift detector was fed one-sided z-scores calibrated only for spread, so it reported a constant trend and flooded the system with false alarms; a weekly-delta feature with too little history and an out-of-range term in Tier 2 each later caused regressions. Each fix was kept only when it improved the incident-level metrics.

TABLE V
INCIDENT-LEVEL PRECISION AND RECALL ACROSS ITERATIONS

Change	Precision	Recall
Initial ensemble	0.10	0.67
Two-sided, out-of-sample drift calibration	0.75	1.00
Sequence attribution; shorter idle timeout	0.31*	1.00
Weekly delta removed; Tier-2 range term added	0.29*	0.42
Drop Tier-2 range term; sparse-feature guard	0.88	1.00
Freeze drift EWMA on acute windows	0.91	1.00

* Regressions: the third iteration exposed a weekly-delta baseline formed online from too few samples; the fourth was caused by the Tier-2 out-of-range term. Each was diagnosed and fixed in the next step.

VII. LESSONS FROM THE FULL-STACK DEPLOYMENT

The single-process stack passed all tests, yet running the 20-service container stack (Kafka, ClickHouse, object storage, MLflow with PostgreSQL, Prometheus, Alertmanager, Grafana and one container per stage) exposed fifteen defects. The most instructive were:

Write patterns. The storage sink inserted one batch per poll; during a history replay this created a MergeTree part per poll and ClickHouse spent its CPU on merges until the container host failed. Buffering across polls and committing offsets only after the write raised ingestion from 127 to about 2,500 records/s. The archive sink similarly committed offsets before data reached object storage.

Clock steps. Kafka 4.0 and 4.1 group coordinators failed offset commits ("histogram recorded value cannot be negative") when the virtual machine's clock stepped backwards after host sleep, which stalled every consumer group. Inspecting the coordinator bytecode showed that 4.3.0 clamps the value, and upgrading resolved it. A health check that launched a JVM every 10 s starved the single broker until it was fenced by its own controller, which in turn left idempotent producers in a fatal state.

Operational semantics. Alertmanager expires alerts that are not re-sent, so the router must re-send open alerts periodically; a router restart must restore open alerts to avoid duplicates; and a dead-man's-switch rule must aggregate across stages. None of these appear without the real components.

A further operational lesson concerns resources: on a laptop virtual machine with 3.7 GB of memory, training alongside the full stack exhausted memory, which supports the proposal's request for a dedicated server (8+ vCPU, 32 GB) for staging.

VIII. LIMITATIONS AND THREATS TO VALIDITY

The workload is simulated. Although it includes seasonality, noise, benign deployments and realistic message formats, real services exhibit more template diversity and messier failures, and incidents overlap. The evaluation covers 12 and 18 incidents, so per-fault estimates rest on two or three cases each. The sequence model was not yet validated on the public HDFS and BGL benchmarks [6], for which a harness is provided. The Flink jobs, the Kubernetes deployment and the CI workflows were written against documented interfaces but not executed, and external alert channels (Slack, PagerDuty) were not connected. Finally, the static baselines are simple; a comparison with tuned per-service thresholds or a commercial tool would strengthen the conclusions.

IX. CONCLUSION AND FUTURE WORK

We designed, implemented and evaluated a streaming log anomaly detection platform that learns normal behaviour per service and hour, combines statistical, isolation-forest and sequence models with hard rules, and delivers few, grouped and explained alerts. On labelled fault injection it met every target of the project proposal—precision 0.91, recall 1.00, 2.3 alerts per service per day and a 95th-percentile latency of 37.6 s—where the static rule in use today detected nothing. Next steps are an evaluation on an anonymised sample of real logs, validation of the sequence model on HDFS and BGL, deployment of the Flink jobs and Helm chart to a staging cluster with load and chaos tests at twice the target rate, and, from accumulated feedback, a supervised re-ranker and LLM-generated incident summaries.

REFERENCES

[1] P. He, J. Zhu, Z. Zheng, and M. R. Lyu, "Drain: An online log parsing approach with fixed depth tree," in *Proc. IEEE Int. Conf. Web Services (ICWS)*, 2017, pp. 33–40.

[2] M. Du, F. Li, G. Zheng, and V. Srikumar, "DeepLog: Anomaly detection and diagnosis from system logs through deep learning," in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, 2017, pp. 1285–1298.

[3] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proc. IEEE Int. Conf. Data Mining (ICDM)*, 2008, pp. 413–422.

[4] S. He, J. Zhu, P. He, and M. R. Lyu, "Experience report: System log analysis for anomaly detection," in *Proc. IEEE Int. Symp. Software Reliability Engineering (ISSRE)*, 2016, pp. 207–218.

[5] W. Xu, L. Huang, A. Fox, D. Patterson, and M. I. Jordan, "Detecting large-scale system problems by mining console logs," in *Proc. ACM Symp. Operating Systems Principles (SOSP)*, 2009, pp. 117–132.

[6] J. Zhu, S. He, J. Liu, P. He, Q. Xie, Z. Zheng, and M. R. Lyu, "Tools and benchmarks for automated log parsing," in *Proc. IEEE/ACM Int. Conf. Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2019, pp. 121–130.

[7] J. Kreps, N. Narkhede, and J. Rao, "Kafka: A distributed messaging system for log processing," in *Proc. NetDB Workshop*, 2011.

[8] P. Carbone, A. Katsifodimos, S. Ewen, V. Markl, S. Haridi, and K. Tzoumas, "Apache Flink: Stream and batch processing in a single engine," *IEEE Data Eng. Bull.*, vol. 38, no. 4, pp. 28–38, 2015.

[9] P. J. Rousseeuw and C. Croux, "Alternatives to the median absolute deviation," *J. Amer. Statist. Assoc.*, vol. 88, no. 424, pp. 1273–1283, 1993.

[10] S. He, P. He, Z. Chen, T. Yang, Y. Su, and M. R. Lyu, “A survey on automated log analysis for reliability engineering,” *ACM Comput. Surv.*, vol. 54, no. 6, pp. 1–37, 2021.

[11] B. Beyer, C. Jones, J. Petoff, and N. R. Murphy, Eds., *Site Reliability Engineering: How Google Runs Production Systems*. Sebastopol, CA, USA: O’Reilly Media, 2016.

[12] T. Akidau *et al.*, “The dataflow model: A practical approach to balancing correctness, latency, and cost in massive-scale, unbounded, out-of-order data processing,” *Proc. VLDB Endow.*, vol. 8, no. 12, pp. 1792–1803, 2015.

[13] M. Zaharia *et al.*, “Accelerating the machine learning lifecycle with MLflow,” *IEEE Data Eng. Bull.*, vol. 41, no. 4, pp. 39–45, 2018.

[14] J. Lin, “Divergence measures based on the Shannon entropy,” *IEEE Trans. Inf. Theory*, vol. 37, no. 1, pp. 145–151, 1991.